SNAPASSIST

AI-POWERED PHOTOGRAPHY & VIDEOGRAPHY
CUSTOMER ENQUIRY AND BOOKING ASSISTANT

COMPLETE PROJECT SPECIFICATION + STEP-BY-STEP BUILD GUIDE

Prepared by: Rakavi
Purpose: Mentor review, implementation, testing and real-studio pilot

AI supports enquiries; the studio owner controls final price, availability and booking approval.

DOCUMENT STATUS

This is a build-ready plan, not a claim that the application is already implemented. Mark features complete only after coding and testing. Sample names and dates are fictional.

CONTENTS

How to Use This Guide

PART A — PRODUCT SPECIFICATION

Business problem, scope, user journeys, requirements, architecture and data model.

PART B — STEP-BY-STEP IMPLEMENTATION

Environment, website, database, Flask routes, AI, owner dashboard, WhatsApp and deployment.

PART C — TESTING & PILOT LAUNCH

Test cases, security, deployment and real-studio validation.

APPENDIX

Suggested folder structure, AI rules, demo script and completion checklist.

Recommended build order: reliable enquiry form and database first; AI conversation second; owner controls third; messaging automation last.

1. Product Overview

SnapAssist is a proposed web-based AI assistant for photography and videography studios. Customers reach a studio-branded page through a website link, Instagram, WhatsApp, Facebook, Google Business Profile or QR code. They can ask about services, share event requirements and submit an enquiry. The owner reviews the request, discusses final price and availability, then confirms or declines the booking.

Target services

- Candid photography
- Pre-wedding and post-wedding shoots
- Outdoor photography
- Baby shower
- Naming ceremony
- Puberty ceremony
- Photography + videography packages

Business problem

Studios receive repeated questions about services, package inclusions, event dates, location charges and booking rules. Owners may be busy at shoots, while customer requirements can change across messages. SnapAssist organises initial enquiries and makes each request traceable.

Product boundary

> **AI handles first-level questions and structures information. It must not invent prices, promise availability, collect payment or confirm a booking on its own. Final price, date availability, package changes and confirmation remain under owner control.**

2. Scope and Requirements

ID	Feature	Acceptance criteria	Priority
FR-01	Studio-branded page	Approved studio name, logo, services and contact details.	Must
FR-02	Service/FAQ assistant	Uses owner-approved content; unknown details are escalated.	Must
FR-03	Enquiry collection	Collects name, contact, service, date, location and requirements.	Must
FR-04	Enquiry reference	Creates a unique ID and saves the record.	Must
FR-05	Owner dashboard	Owner securely views and updates enquiries.	Must
FR-06	Change requests	Package/date/location/service changes are tracked, not auto-confirmed.	Must
FR-07	Human confirmation	Only authorised owner can set Booking Confirmed.	Must
FR-08	WhatsApp handoff	Prepared enquiry message opens in WhatsApp.	First release
FR-09	Automatic messages	Customer and owner notified after owner confirmation via approved API.	Later phase
FR-10	Multi-studio	Separate branding, packages, contacts and enquiry data per studio.	Future

Non-functional requirements

- Mobile-first design and a clear human-contact option.
- Keep API keys server-side in environment variables.
- Require owner login and role checks for dashboard routes.
- Validate input and preserve change history.
- Failed messages must not erase a confirmed booking.
- Explain why customer details are collected and how correction/deletion can be requested.

3. End-to-End Workflow

Customer opens studio link	**AI answers basic questions**	**Customer shares event details**	**System saves Enquiry ID**	**Owner reviews and contacts**
Customer requests package/date change	**System records change request**	**Owner checks price + date**	**Customer agrees to revised terms**	**Owner confirms; messages triggered**

Example

A customer initially asks for Premium Pre-Wedding, then requests Basic. The system preserves the original package and records Basic as a requested change. The owner checks price and availability, discusses revised terms, and confirms only after agreement.

Returning customer

Use an enquiry ID with a verification step or a secure link sent to a verified contact. An easily guessed ID alone must not expose personal enquiry details.

4. Enquiry Status Model

Status	Meaning	Who controls it
New Enquiry	Initial details submitted.	System
Waiting for Owner	Saved and needs review.	System
Owner Contacted	Owner contacted customer.	Owner
Change Requested	Customer requested a change.	Request recorded; owner reviews
Awaiting Customer Decision	Owner sent revised options/price.	Owner
Awaiting Owner Confirmation	Customer accepts proposed terms; owner must approve.	Workflow/owner
Booking Confirmed	Owner approved date, service, package and price.	Owner only
Cancelled / Closed	Enquiry cancelled or closed with reason.	Owner; record reason

Keep an audit trail for material changes: field, previous value, new value, timestamp and actor. Booking status and message-delivery status are separate fields.

5. Architecture and Technology

Component	Purpose	Recommended choice
Frontend	Website, chat, forms, owner dashboard.	HTML, CSS, JavaScript, Jinja
Backend	Routes, validation, rules, authentication.	Python + Flask
AI	Natural-language replies grounded in studio data.	OpenAI/ChatGPT API via backend
Database	Studios, services, packages, enquiries, users, history.	SQLite locally; PostgreSQL for production
Messaging	Prepared WhatsApp enquiry link first; automated messages later.	WhatsApp click-to-chat then Business Platform/API
Hosting	Public HTTPS website and environment settings.	Render or suitable cloud host

Data flow

Customer browser	**Flask backend**	**AI API + approved studio content**	**Database**	**Owner dashboard / WhatsApp**

Technical rules

- Never place the private AI API key in browser code.
- Ground replies in the selected studio's approved FAQs and package data.
- If price or availability is missing, say the owner must confirm it.
- For multiple studios, scope every database query to the correct studio.
- Click-to-chat opens a prepared message; automated outbound messages require approved API setup and platform compliance.

6. Database Design

Table	Example fields	Purpose
studios	id, name, logo, phone, location, hours	Studio profile and contacts.
users	id, studio_id, email, password_hash, role	Owner/admin login.
services	id, studio_id, name, description, active	Services offered.
packages	id, studio_id, service_id, name, price, inclusions	Owner-managed packages.
enquiries	id, public_reference, studio_id, customer_name, phone, service, event_date, location, status	Current request.
enquiry_history	id, enquiry_id, field, old_value, new_value, actor, timestamp	Change audit trail.
notifications	id, enquiry_id, channel, status, provider_reference, timestamp	Message attempts and status.

Minimum enquiry fields

- Unique reference such as SA-2026-00125, generated by backend.
- Customer name and contact method, with a clear privacy notice.
- Service type, event date, location, package preference, photo/video needs.
- Additional notes and preferred contact time.
- Status, owner notes, final package, final price and confirmation timestamp.

7. Step-by-Step Build Plan

Step 1 — Freeze the MVP scope

Build one studio first: branded site, FAQ chatbot, enquiry save, unique ID, owner dashboard, owner-controlled status updates and WhatsApp click-to-chat. Delay multi-studio accounts, payments and automated notifications.

Step 2 — Prepare studio information

Get permission for logo/photos. Prepare services, packages, inclusions, FAQs, location, phone, business hours, booking rules and cancellation terms. Missing price/availability must be marked for owner confirmation.

Step 3 — Set up Python environment

Create the project folder and virtual environment. Install Flask, the current OpenAI Python SDK and database tools. Create requirements.txt. Put API keys in .env and add .env to .gitignore.

Step 4 — Build the responsive website

Create homepage, services/portfolio, chatbot panel, enquiry form and contact button. Test mobile layout. Use placeholder branding until the actual owner approves assets.

Step 5 — Create database models

Create studios, users, services, packages, enquiries and enquiry_history. Generate unique references. Save the record before showing a success message.

Step 6 — Build Flask routes

Implement homepage, service list, enquiry submission, permitted enquiry view, owner login, owner enquiry list and owner status-update routes. Validate data server-side.

Step 7 — Add AI API

Call the AI API from Flask only. Supply approved studio FAQs/packages. Instruct it not to invent price, availability or confirmation. Add error fallback and human-contact option.

Step 8 — Build owner dashboard

Require login. Display enquiry ID, contact, service, date, location, package and status. Owner can add notes, contact customer and propose revised details.

Step 9 — Handle changes

When a customer requests a package/date/location change, save it as Change Requested and preserve the previous value. Owner checks details and only confirms after agreement.

Step 10 — WhatsApp handoff

Start with a click-to-chat link and pre-filled enquiry summary. The user must press Send. Test the phone number format and message content.

Step 11 — Automatic notifications

Only after the core workflow works, configure WhatsApp Business Platform/API or supported provider. Verify account setup, templates/consent, callbacks, retries and failure logs.

Step 12 — Test and deploy

Run the test plan. Configure production secrets, HTTPS, persistent database, backups, access control and logs. Deploy and test the public link on mobile.

8. Suggested Project Structure

```
SnapAssist/
  app.py
  config.py
  models.py
  requirements.txt
  .env                # local secrets; never commit
  .gitignore
  templates/
    base.html
    index.html
    services.html
    chat.html
    enquiry_success.html
    owner_login.html
    owner_dashboard.html
    enquiry_detail.html
  static/
    css/style.css
    js/chat.js
    images/
  instance/
    snapassist.db      # local development database
  tests/
    test_enquiries.py
    test_booking_status.py
  README.md
```

Suggested Flask routes

Route	Purpose
GET /	Studio landing page.
GET /services	Studio-approved services and packages.
POST /api/chat	Send message to backend AI logic.
POST /enquiries	Validate and save enquiry; return reference.
GET /enquiries/<reference>	Show enquiry only after proper verification.
GET /owner/login	Owner login page.
GET /owner/enquiries	Owner-only enquiry list.
GET /owner/enquiries/<id>	Owner-only enquiry details.
POST /owner/enquiries/<id>/update	Record changes/status and history.
POST /owner/enquiries/<id>/confirm	Owner confirms; trigger notifications.

9. AI Behaviour and Prompt Rules

Suggested system instructions

You are the customer enquiry assistant for the selected photography studio. Answer politely using only studio-approved service, package and FAQ information supplied by the backend. Ask only for details needed to answer or create an enquiry. Never invent prices, package inclusions, availability, discounts, payment status or booking confirmation. If information is missing or requires business approval, say the studio owner will confirm it. For changes to an existing enquiry, use the application's approved verification flow and create a change request instead of confirming the change. Be transparent that you are an AI assistant. Offer a way to contact the studio.

Example conversation

Actor	Example
Customer	I need pre-wedding photography and video in December.
Assistant	I can collect your requirements. Which date and location are you considering? The studio will confirm availability and final pricing.
Customer	I first selected Premium. Can I change to Basic?
Assistant	I can record that change request. Please use your enquiry reference and verification flow. The owner will confirm revised package, price and date.
System	Preserve Premium as original; save Basic as requested package; set Change Requested.
Owner	Checks price/calendar, agrees details with customer and confirms.
System	After owner confirmation, update status and trigger configured notifications.

10. WhatsApp and Confirmation Design

Release 1 — click-to-chat

A button opens WhatsApp with a pre-filled enquiry summary. The customer or owner still presses Send. This is the simplest first release and should not be described as automatic messaging.

Release 2 — automated notifications

- Use WhatsApp Business Platform/API or a supported provider, not unofficial personal-account automation.
- Store tokens securely on the server.
- Verify current provider, message-template and consent requirements before launch.
- Trigger messages only after owner-approved status change.
- Record queued/sent/delivered/failed states where the provider supplies them.
- On failure, show retry/manual-contact options; do not reverse the booking.

Customer confirmation template

Hello [Customer Name], your booking has been confirmed by [Studio Name].
Booking ID: [Reference]
Service: [Service]
Package: [Final Package]
Date: [Confirmed Date]
Location: [Confirmed Location]
Final amount: [Owner-confirmed amount]
Contact the studio if any detail needs correction.

Owner notification template

Booking confirmed — [Reference]. Customer: [Name]. Service: [Service]. Package: [Final Package]. Date: [Date]. Location: [Location]. Amount: [Final amount]. Customer notification status: [Status].

11. Testing Checklist

Test scenario	Expected result
Question about listed service	Bot gives approved service details.
Question about unpublished price	Bot does not invent price; escalates to owner.
Customer submits enquiry	Database record and unique reference are created.
Database save fails	No false success message; retry/fallback shown.
Customer changes package	Original value preserved; Change Requested recorded.
Customer tries to view another enquiry	Access denied without verification.
Owner confirms with missing fields	System blocks confirmation and explains why.
Owner confirms final booking	Status changes and configured notification starts.
Messaging API fails	Failure logged; owner can retry or contact manually.
Unauthenticated visitor opens owner route	Access denied or redirected to login.
Studio A tries to view Studio B data	Access denied in multi-studio version.
Mobile browser test	Chat, form and dashboard remain usable.

Minimum definition of done

- Enquiry can be created, stored and retrieved.
- Owner can securely review and update enquiry.
- Package/date changes cannot auto-confirm booking.
- Only owner can confirm final booking.
- AI fallback prevents fabricated price/availability.
- WhatsApp behaviour is tested and labelled manual vs automatic.
- Secrets are excluded from repository; deployment and backups are verified.

12. Security, Deployment and Pilot

Security checklist

- Keep .env, credentials and customer data out of version control.
- Hash passwords; never store plain-text passwords.
- Validate inputs and use ORM/parameterised database operations.
- Use HTTPS and restrict owner routes.
- Limit AI context; do not give it unrestricted database actions.
- Do not expose enquiry details based on a guessable ID alone.
- Back up production data and define retention/deletion rules.

Deployment sequence

- Review repository for secrets before pushing code.
- Configure production environment variables at the host.
- Use a persistent production database; do not assume a local SQLite file survives every redeploy.
- Test HTTPS, database connection, AI failures and owner login.
- Test WhatsApp with approved test recipients/templates before launch.

Pilot with one studio

Obtain permission for logo, portfolio and business information. Run test enquiries first. Ask the owner to verify summaries, change requests and confirmation controls. Collect feedback and fix issues before sharing the public link.

APPENDIX

13. Mentor Demo Plan and Build Order

Time	Demonstration
0:00–0:40	Explain the enquiry problem and human-in-the-loop design.
0:40–1:30	Open branded website and ask about pre-wedding services.
1:30–2:15	Submit enquiry and show unique reference ID.
2:15–3:00	Open owner dashboard and review saved enquiry.
3:00–3:45	Change Premium to Basic; show Change Requested status.
3:45–4:30	Owner checks details and confirms; show notification/log behaviour.
4:30–5:00	Summarise safeguards, tests and next steps.

Build stages

Stage	Focus	Move forward when...
1	Website + enquiry form	Validation and submission work.
2	Database + reference ID	Records persist after restart.
3	Owner login/dashboard	Owner securely reviews enquiries.
4	Status + change history	Changes never silently confirm bookings.
5	AI API	Grounded responses and fallback work.
6	WhatsApp handoff	Prepared message is correct.
7	Automatic notifications + deployment	API, secrets, logs and failures are tested.

APPENDIX

14. Glossary and Final Checklist

Term	Meaning
AI assistant	Conversational interface using approved studio information.
Enquiry ID	Unique request reference; not proof of identity by itself.
Human-in-the-loop	AI assists, but a person approves important decisions.
Click-to-chat	Opens WhatsApp with prepared text; user sends it manually.
WhatsApp Business API	Business messaging integration subject to provider/platform rules.
Audit trail	History of who changed what and when.
MVP	Smallest useful version tested with a real studio.

Before calling the product ready

- Studio approved its branding and content.
- Enquiry saving and unique ID tested.
- Owner authentication and permissions tested.
- Package/date changes and audit history work.
- Only owner can confirm booking.
- AI does not guess prices or availability.
- WhatsApp behaviour matches what the UI promises.
- Privacy, secrets, backups and error handling checked.

Final note

SnapAssist becomes a real business product through implementation, testing and feedback from a real studio—not documentation alone. Build the enquiry and owner-approval workflow first, then add AI and automated messaging in controlled stages.

Prepared for mentor review and implementation. Replace sample information with owner-approved details before production.